

Course Materials for NLP Term Project: Building a Coptic–Syriac Neural Machine Translator: A Term Project Tutorial on Extending Multilingual Pre-trained Models to Low-Resource Ancient Languages

Prof. Moon-Ryul Jung
Dept of Art & Technology
Sogang University

1 Project Overview and Motivation

This term project asks you to build a neural machine translation (NMT) system between two low-resource ancient languages: **Coptic** (the language of late ancient Egypt, written in a Greek-derived alphabet) and **Syriac** (an Eastern Aramaic language used by Syriac Christianity, written in its own alphabet). Both languages are crucial for biblical studies, patristic scholarship, and the digital humanities, yet neither is covered by mainstream multilingual NMT systems such as NLLB-200 [6].

The project will introduce you to the entire pipeline of modern multilingual NMT: tokenizer extension, embedding initialization, pre-trained model selection, multi-task training, back-translation [10], and evaluation on low-resource pairs. By the end, you will have built a working translation system and developed a deep understanding of *why* the choices made along the way matter.

The deeper pedagogical goal is to confront the central problem of low-resource NLP: **how do we transfer knowledge from richly-resourced models to languages they have never seen**, when the very vocabulary, scripts, and subword units of those languages are unknown to the model?

1.1 What You Will Build

A bidirectional translation system (Coptic $\leftrightarrow$ Syriac) based on extending a pre-trained multilingual model. You will choose between two architectural starting points:

1. **Glott500-m encoder + new decoder** [5]: An encoder-only multilingual model (covering 511 languages) to which you add a translation decoder.
2. **NLLB-200 fine-tuning** [6]: An existing encoder-decoder translation model (covering 200 languages) which you extend with new vocabulary for Coptic and Syriac.

Section 3 will help you make this choice; both are valid and the comparison is itself a key learning outcome.

1.2 Project Deliverables

- A working translation model (Coptic $\rightarrow$ Syriac and Syriac $\rightarrow$ Coptic).
- Quantitative evaluation on a held-out test set using BLEU [17], chrF++ [18], and qualitative analysis.
- A 10-page written report covering: design choices, data collection, training pipeline, results, and ablation studies.
- Released code on GitHub, with reproducibility instructions.

1.3 Computational Resources

This project is designed for a setup of **4 NVIDIA RTX A6000 GPUs** (each with 48 GB VRAM, for a total of 192 GB). This is sufficient for full fine-tuning of NLLB-200-distilled-1.3B or Glot500-m with a custom decoder, and for multi-GPU training using FSDP [16] or DeepSpeed ZeRO [15] if needed.

2 Background: Why This Is Hard

2.1 The Vocabulary Bottleneck

Modern multilingual models such as mBERT [3], XLM-R [4], NLLB [6], and Glot500 [5] rely on subword tokenization (BPE [7] or SentencePiece [9] with UnigramLM [8]). The tokenizer’s vocabulary V is learned once during pretraining from a corpus dominated by a small number of high-resource scripts (primarily Latin). The result is what [1] call “the curse of mBERT”: languages whose scripts are underrepresented in the pretraining corpus are tokenized into meaningless character-level fragments, with their embeddings becoming language-agnostic averages incapable of supporting downstream tasks.

2.2 How Embeddings Are Learned: Language-Agnostic Averages

The phrase “language-agnostic averages” is a compressed description of an important phenomenon in how multilingual model embeddings are formed during training. Let us unpack exactly what it means by tracing through the mechanism step by step.

During training, the embedding $E[w]$ for any token w receives gradient updates every time that token appears in the training data. At each occurrence, the gradient reflects:

- The context in which the token appears.
- The model’s prediction loss at that step.
- The role the token plays in that particular sentence.

Over many occurrences across the entire training corpus, $E[w]$ becomes essentially the **accumulated result of all the gradient contributions from every use of that token**. Informally, we can express this as:

$$E[w] \approx \frac{1}{N} \sum_{i=1}^N (\text{gradient contribution from occurrence } i) \quad (1)$$

where N is the total number of times the token appeared during training. The final embedding is, in a meaningful sense, an average over all the ways the token was used.

2.3 When a Single Character Appears Across Multiple Languages

This is where “language-agnostic” enters the picture. Let us work through a concrete example.

Consider the Arabic letter sheen. During mBERT’s training on Wikipedia data from 104 languages, this character appears in many different language contexts:

- In **Standard Arabic** text, it occurs tens of millions of times, playing specific morphological roles within Arabic’s triconsonantal root system.
- In **Persian** text, it occurs millions of times, but Persian is a completely different language family (Indo-European, not Semitic), so the letter plays a different role in word formation.
- In **Urdu** text (also Indo-European), the character participates in yet another set of morphological patterns.
- In **Pashto, Sindhi, Uyghur, Kurdish**, and other languages that use Perso-Arabic script, the same character appears in even more distinct linguistic contexts.

Each language uses this character differently. The character carries different phonological associations, appears in different positions within morphological structures, and combines with other characters according to different grammatical rules across these languages.

2.4 The Averaging Effect

Crucially, mBERT does not maintain separate embeddings for “sheen in Arabic” versus “sheen in Persian” versus “sheen in Urdu.” There is only **one embedding** $E[\text{sheen}]$ in the model. Every occurrence of this character, regardless of which language it appears in, contributes gradients to this single shared embedding.

The result is that $E[\text{sheen}]$ ends up as an average over all these uses:

$$E[\text{sheen}] \approx \frac{(\text{gradients from Arabic}) + (\text{gradients from Persian}) + (\text{gradients from Urdu}) + \dots}{\text{total occurrences}} \quad (2)$$

This average is **not optimized for any specific language**. It represents a kind of “lowest common denominator” across all the languages where the character appears.

2.5 Why This Is Called “Language-Agnostic”

The term “agnostic” here means “without knowledge of” or “not committed to.” The embedding $E[\text{sheen}]$ is language-agnostic in the sense that:

- It does not encode the specific morphological patterns of Arabic.
- It does not encode the specific morphological patterns of Persian.
- It does not encode the specific morphological patterns of Urdu.
- Instead, it encodes a vague statistical summary of how the character is used **across all these languages collectively**.

For any specific downstream task in any specific language (say, POS tagging in Uyghur or NER in Sorani Kurdish), this embedding is poorly suited. It is too general and too diluted to provide the rich linguistic signal that the task requires.

2.6 The Direct Answer to “Average of What?”

So when the tutorial says embeddings become “language-agnostic averages,” the precise content of the average is:

The average over the gradient contributions accumulated from every occurrence of that token across every language in the training corpus where the token appears.

The constituent elements being averaged are:

- Each individual occurrence of the token in training data.
- These occurrences are drawn from many different languages.
- Each language uses the token in different ways with different meanings and roles.
- All of these contributions flow into the same single embedding vector.
- The final result is a generalized average that is not specialized for any particular language.

2.7 Contrast: Language-Specific Embeddings

To make this clearer by contrast, consider what a “language-specific” embedding looks like.

Language-specific example: the English suffix `##tion` in monolingual English BERT

- The token appears only in one language (English).
- All its occurrences play essentially the same morphological role (forming nouns from verbs: *creation*, *formation*, *education*, *recognition*).
- All gradient updates push the embedding in a consistent direction.
- The result is a sharp, well-defined embedding that strongly encodes the concept of “nominalization.”

When a downstream model processes English text containing `##tion`, this embedding immediately signals “this is a noun derived from a verb,” which is rich and useful information.

Language-agnostic example: a single Arabic character in mBERT

- The token appears across many languages.
- Each language uses it differently.
- Gradients flow in inconsistent directions during training.
- The result is a diffuse, ambiguous embedding that primarily reflects “this character exists and gets used somehow in several languages.”

When a downstream model processes Uyghur text containing this character, the embedding provides almost no language-specific information. The model is essentially starting from a near-blank slate for this character in the Uyghur context.

2.8 A Second Sense of “Average”: Context Averaging

There is also a second, related sense in which these embeddings are “averages,” which applies even within a single language.

When a token is very short (like a single character) and appears in an enormous variety of contexts within a single language, its embedding becomes an average over all those contexts as well.

For example, the letter **e** in English:

- Appears in nearly every English word.
- Plays no specific morphological or semantic role on its own.
- Its embedding ends up being an average over thousands of unrelated contexts.
- The result is an embedding that says “this is a very common English character” but carries no specific meaning.

By contrast, a longer subword like **##tion**:

- Appears only in specialized contexts (verbal nouns).
- Its embedding is averaged over a much narrower set of contexts.
- The result is a more focused, semantically meaningful representation.

2.9 The Combined Effect

The two senses of “average” compound when we consider multilingual models handling underrepresented scripts. A single Arabic character in mBERT is simultaneously:

1. **Averaged across many languages** (Arabic, Persian, Urdu, Uyghur, Sindhi, and so on), so it is not specific to any one language.
2. **Averaged across many contexts within each language** (since single characters appear everywhere in their respective languages).

The double averaging produces an embedding that is essentially statistical noise from any specific language’s perspective. The character is “in” the model, but its representation carries vanishingly little linguistic information that would be useful for downstream tasks in Uyghur or any other specific language.

2.10 Why This Matters

The phenomenon described above — where a token’s embedding becomes a diffuse average across the many languages and many contexts in which it appears during training — **is precisely the mechanism** behind what Muller et al. [1] call the failure mode of mBERT on unseen languages. The effect is most severe for single-character tokens, which are precisely what an underrepresented script gets reduced to during BPE tokenization. When an Uyghur word gets tokenized into individual Arabic characters (because mBERT’s vocabulary has no Uyghur-specific subwords), each character contributes only a language-agnostic average embedding to the model’s representation of that word. The model receives a sequence of poorly-informed tokens rather than meaningful subword units, and downstream tasks fail because the input representation is too impoverished to support them.

The averaging phenomenon applies to *any token* whose embedding gets diluted across many languages or many contexts, not just to single characters. Let us distinguish three cases:

Single-character tokens: This is the worst case where averaging is most severe. When a script is underrepresented in BPE training, multi-character merges never form for that script, so the vocabulary contains only individual characters for that script. These individual characters appear across many languages and contexts, producing the most diluted embeddings.

Short multi-character tokens: Less severe but still affected. A 2–3 character subword in an underrepresented script might appear in a few languages with different roles. Some averaging still happens, just less catastrophically.

Longer, language-specific tokens: Best case — not affected by the phenomenon. A long subword like `##tion` appears only in English contexts where it has a consistent role. Its embedding is sharp and well-defined.

So the correct framing is that the averaging is a property of how any token is used during training, but the single-character case is the extreme version that occurs in practice for underrepresented scripts.

The tutorial uses the phrase “language-agnostic averages” as a compact way to describe this entire phenomenon. A more verbose but clearer alternative would be:

“...their embeddings become averages over the token’s usage across many unrelated languages and across many contexts within each language, producing representations too diffuse to encode the language-specific morphological or semantic information needed for downstream tasks.”

This is the precise mechanism by which the vocabulary mismatch problem translates into actual representational failure in the model. Vocabulary extension addresses this problem by creating new tokens that are used only in the new languages, so that their embeddings can be specialized rather than averaged across many unrelated uses.

2.11 The Specific Challenge for Coptic and Syriac

For Coptic (using the Coptic alphabet, Unicode block U+2C80–U+2CFF) and Syriac (using the Syriac alphabet, Unicode block U+0700–U+074F), this problem is acute. Neither script is well-represented in the SentencePiece [9] vocabulary of mainstream models. Your project must therefore

confront the question: **how do we extend V to cover new scripts, and how do we initialize the corresponding embeddings so the model can actually use them?**

2.12 The Data Scarcity Problem

Coptic and Syriac are *ancient* low-resource languages, which compounds the typical challenges of low-resource NMT:

- **Domain restriction:** Most surviving text is religious or classical (Bibles, patristic literature, liturgy). Modern domain texts are essentially absent.
- **Direct parallel data nearly nonexistent:** Coptic–Syriac direct translations are vanishingly rare. You will likely have only a few hundred sentences at most.
- **Auxiliary parallel data exists:** Coptic–Greek, Syriac–Greek, Coptic–English, Syriac–English, Coptic–Arabic, and Syriac–Arabic alignments are available, primarily through Bible translations and patristic studies.
- **Monolingual data is moderate:** Tens of thousands to hundreds of thousands of sentences in each language are accessible through digitized collections.

This data profile shapes everything about how you will design training. You cannot rely on direct Coptic–Syriac parallel data alone; you must exploit cross-lingual transfer through pivot languages (English, Greek, Arabic) and leverage monolingual data through self-supervised objectives and back-translation [10].

2.13 Key Insight: The Transformer’s Structure Helps

One critical observation that motivates everything that follows: in a Transformer [2], the attention projection matrices (W_Q, W_K, W_V, W_O) and the FFN matrices are **applied identically to every token**, independent of token identity. Their dimensions depend on d_{model} but not on $|V|$. This means adding new tokens to the vocabulary does *not* require modifying these matrices; they automatically operate on the new token’s embeddings just as they did on the old. The only architectural components that change with vocabulary size are the input embedding matrix $E \in \mathbb{R}^{|V| \times d_{\text{model}}}$ and the output projection $W_{\text{out}} \in \mathbb{R}^{d_{\text{model}} \times |V|}$ (which is often tied to E).

This is the structural property that makes vocabulary extension feasible at all: we extend two matrices, and the rest of the model adapts through standard backpropagation.

3 Design Choice: Glot500 vs. NLLB

Your first major design decision is which pre-trained model to extend. Both have legitimate merits; understanding the trade-offs is part of the project. Note that practically speaking, neither model supports Coptic and Syriac.

3.1 Option A: Glot500-m Encoder + New Decoder

Glot500-m [5] is an extension of XLM-R-base [4] (encoder-only) that was continued-pretrained on Glot500-c, a corpus covering 511 languages. Its key property is that it covers far more low-resource languages than any other pre-trained model.

Advantages:

- Smaller model (~395M parameters), comfortable for 4×A6000 with full fine-tuning.
- Cleaner research design: you control exactly what the decoder learns.
- Encoder representations are multilingual and likely cross-lingually aligned.

Disadvantages:

- No pre-trained decoder; you must train one from scratch or initialize it from encoder weights.
- Cross-attention is initialized from random and must be trained, which requires substantial parallel data.
- No prior translation capability to leverage.

3.2 Option B: NLLB-200 Fine-tuning with Vocabulary Extension

NLLB-200 [6] is a Mixture-of-Experts or dense encoder-decoder model pretrained for translation across 200 languages.

Advantages:

- Pre-trained decoder with strong general translation capability.
- Cross-attention already trained on diverse language pairs.
- Cross-lingual transfer from related languages (Greek, Arabic) may help.
- Distilled variants (600M, 1.3B) fit comfortably on 4×A6000.

Disadvantages:

- NLLB’s pre-trained knowledge about Coptic/Syriac is essentially nonexistent, so the size advantage is partially wasted.
- Larger model means slower iteration during development.

3.3 Comparison Summary

Aspect	Glott500 + new decoder	NLLB-200
Decoder pretrained	No	Yes
Cross-attention pretrained	No	Yes
Model size	~395M	600M – 1.3B
Vocabulary extension needed	Yes	Yes
Training compute	Lower	Higher
Direct translation prior	None	Strong (for 200 langs)
Suitability for our task	High	Moderate

Table 1: Comparison of base model options.

3.4 Our Recommendation

NLLB’s pre-trained translation capability is valuable only when the target language is in its 200 supported languages. For genuinely unseen languages, you are extending the model from scratch in either case, and the smaller model (Glot500) has a practical advantage.

4 Vocabulary Extension: Detailed Procedure

This section details exactly what you must do to extend the model’s vocabulary to handle Coptic and Syriac. We focus on Glot500 (which uses XLM-R’s SentencePiece [9] tokenizer) but the procedure is essentially identical for NLLB.

4.1 Step 1: Verify Current Coverage

Before extending anything, check whether the existing tokenizer handles Coptic and Syriac text reasonably, using the Hugging Face Transformers library [14]:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("cis-lmu/glot500-base")

# Sample Coptic and Syriac text
coptic_sample = "<Coptic text here>"
syriac_sample = "<Syriac text here>"

print("Coptic tokens:", tokenizer.tokenize(coptic_sample))
print("Syriac tokens:", tokenizer.tokenize(syriac_sample))
print("UNK count:", sum(1 for t in tokenizer.tokenize(coptic_sample)
                      if t == tokenizer.unk_token))
```

If the tokenization fragments words into individual characters (or many <unk> tokens), vocabulary extension is necessary.

4.2 Step 2: Train SentencePiece on the Target Languages

Collect monolingual corpora for Coptic and Syriac (see Section 7). Train a new SentencePiece [9] model on this data to learn appropriate subwords:

```
import sentencepiece as spm

# Combine monolingual text from both languages
spm.SentencePieceTrainer.train(
    input='coptic_syriac_corpus.txt',
    model_prefix='copt_syr_tokenizer',
    vocab_size=10000,           # Number of new tokens to learn
    model_type='unigram',     # UnigramLM (Kudo, 2018)
    character_coverage=0.9995,
    normalization_rule_name='nmt_nfkc_cf'
)
```

This produces a new SentencePiece model containing approximately 10,000 tokens specific to Coptic and Syriac, learned using the UnigramLM algorithm [8]. The exact number is a hyperparameter you may tune.

4.3 Step 3: Merge Vocabularies

Now you must merge the new tokens into the existing tokenizer. The cleanest approach uses Hugging Face's `add_tokens` interface [14]:

```
# Extract new tokens from your SentencePiece model
import sentencepiece as spm

sp = spm.SentencePieceProcessor()
sp.load('copt_syr_tokenizer.model')

new_tokens = []
for i in range(sp.get_piece_size()):
    token = sp.id_to_piece(i)
    if token not in tokenizer.get_vocab():
        new_tokens.append(token)

print(f"Adding {len(new_tokens)} new tokens")

# Add to existing tokenizer
num_added = tokenizer.add_tokens(new_tokens)
```

After this step, the tokenizer can encode both Coptic and Syriac at the subword level rather than fragmenting into characters.

4.4 Step 4: Extend the Model's Embedding Matrix

The model's input embedding matrix E must grow to accommodate the new tokens. Hugging Face provides a convenience method:

```
from transformers import AutoModel
import torch

model = AutoModel.from_pretrained("cis-lmu/glot500-base")

# Resize embeddings (extends input embeddings
# and output projection if tied)
model.resize_token_embeddings(len(tokenizer))

print(f"New vocabulary size: {len(tokenizer)}")
print(f"New embedding shape: {model.get_input_embeddings().weight.shape}")
```

4.5 Step 5: Initialize New Embeddings Carefully

This is the most important step. Random initialization is poor; the new embeddings must be placed in a meaningful location in the model's representation space. Use **subword composition initialization**:

```
import torch

embeddings = model.get_input_embeddings().weight.data
old_vocab_size = len(tokenizer) - len(new_tokens)

for i, new_token in enumerate(new_tokens):
    new_token_id = tokenizer.convert_tokens_to_ids(new_token)

    # Decompose new_token into existing subwords/characters
```

```

sub_tokens = []
remaining = new_token.replace('_', '') # Strip SP prefix

for char in remaining:
    if char in tokenizer.get_vocab():
        sub_tokens.append(char)

if sub_tokens:
    sub_ids = [tokenizer.convert_tokens_to_ids(t)
               for t in sub_tokens]
    if tokenizer.convert_tokens_to_ids(t) < old_vocab_size]
    if sub_ids:
        # Initialize as mean of constituent embeddings
        embeddings[new_token_id] = embeddings[sub_ids].mean(dim=0)
    else:
        embeddings[new_token_id] = embeddings[:old_vocab_size].mean(dim=0)
else:
    embeddings[new_token_id] = embeddings[:old_vocab_size].mean(dim=0)

```

This places each new token’s embedding at the centroid of its constituent characters in the original embedding space. The new token starts with a sensible representation rather than random noise.

4.6 Step 6: Output Projection Handling

If the model uses tied weights (most do, including XLM-R [4] and NLLB [6]), Step 5 automatically handles the output projection. To verify:

```

input_emb = model.get_input_embeddings().weight
if hasattr(model, 'get_output_embeddings') \
    and model.get_output_embeddings() is not None:
    output_emb = model.get_output_embeddings().weight
    print(f"Tied: {input_emb.data_ptr() == output_emb.data_ptr()}")

```

If untied, you must manually set the output projection rows for new tokens to match the input embeddings.

4.7 Why This Works: A Mathematical Note

After Step 5, when the model processes a sentence containing a new Coptic or Syriac token, the attention projections W_Q, W_K, W_V [2] (which are unchanged from pretraining) operate on the new token’s embedding just as they do on any other token. Since the new embedding lies near the centroid of related existing embeddings in the representation space, the attention patterns it produces are sensible rather than chaotic. This means the rest of the model—FFN layers, layer norms, attention—can immediately operate on the new tokens and only needs minor adjustments through fine-tuning, rather than learning everything from scratch.

5 Adding a Decoder to Glot500 (Option A)

If you chose Glot500, you now need a decoder. Three strategies exist:

5.1 Decoder Initialization

Option A1: Initialize from encoder weights. Copy the entire encoder (12 layers of XLM-R [4]) and use it as the decoder, changing the attention mask to causal. Add randomly-initialized cross-attention sublayers. This is the recommended starting point.

Option A2: Random initialization. Simpler but requires more data.

Option A3: Use a pretrained decoder from mBART [11] or NLLB [6]. Possible but requires aligning vocabularies, which is complex.

```
from transformers import EncoderDecoderModel # Wolf et al., 2020

# Initialize encoder-decoder model with Glot500 weights for both
model = EncoderDecoderModel.from_encoder_decoder_pretrained(
    "cis-lmu/glot500-base", # encoder
    "cis-lmu/glot500-base" # decoder (cross-attention added)
)
```

The `EncoderDecoderModel` class from Hugging Face Transformers [14] automatically:

- Copies the encoder weights to both sides.
- Switches the decoder’s self-attention to causal mask.
- Adds randomly-initialized cross-attention layers between self-attention and FFN.

5.2 The Cross-Attention Problem

The new cross-attention layers ($W_Q^{\text{cross}}, W_K^{\text{cross}}, W_V^{\text{cross}}$) start from random. They must learn to map encoder representations into the decoder’s processing pipeline [2]. This is where most of the training effort goes.

For cross-attention to learn well, you need diverse (encoder representation, target sequence) pairs—not just Coptic–Syriac, but many parallel pairs involving either language. This is what makes multi-task training essential (Section 6).

6 Training Pipeline

We recommend a five-phase training pipeline.

6.1 Phase 1: Encoder Adaptation (Continued Pretraining)

Goal: Adapt the encoder to Coptic and Syriac through Masked Language Modeling (MLM) [3] training on monolingual data.

Data: All Coptic and Syriac monolingual text you have collected.

Objective: Standard MLM as in BERT [3] and XLM-R [4].

Duration: 1–3 epochs, depending on data size.

Why first: Before the decoder learns to use encoder representations, those representations should already be high-quality for the new languages. This phase ensures new token embeddings are well-integrated into the model’s representation space.

6.2 Phase 2: Multi-task Translation Pretraining

Goal: Train the decoder and cross-attention on diverse parallel data including all available language pairs involving Coptic or Syriac.

Data (Tier 1+2):

- Coptic–English, English–Coptic (Bible, patristic)
- Syriac–English, English–Syriac (Peshitta, patristic)
- Coptic–Greek, Greek–Coptic (Bible alignments)
- Syriac–Greek, Greek–Syriac (Peshitta, Greek patristic)
- Coptic–Arabic, Syriac–Arabic (where available)
- Small samples of high-resource pairs for grounding

Objective: Standard translation cross-entropy.

Sampling: Mix language pairs in each batch. Use temperature-based sampling so low-resource pairs are upsampled, following the approach of NLLB [6].

Duration: The bulk of your training compute, typically 5–20 epochs.

This is where the model actually learns to translate. Cross-attention learns to map encoder representations to target sequences; the decoder learns to generate Coptic and Syriac.

6.3 Phase 3: Coptic–Syriac Specialization

Goal: Specialize the model on the actual Coptic–Syriac direction.

Data: Any direct Coptic–Syriac parallel data you can find. Even a few hundred sentences are valuable.

Strategy: Very small learning rate (e.g., 1e-6); few epochs; monitor validation carefully to avoid overfitting on tiny data.

6.4 Phase 4: Pivot Translation Baseline

Goal: Establish a translation route through a pivot language for cases where direct quality is poor.

Method: Coptic → Greek → Syriac (or via English). The model already learned both legs in Phase 2.

Use: Generate synthetic Coptic–Syriac pairs by piping monolingual Coptic through Coptic→Greek→Syriac.

6.5 Phase 5: Back-translation

Goal: Augment training data using the model itself, following the back-translation technique of Sennrich et al. [10].

Method:

1. Use the current model to translate monolingual Coptic to synthetic Syriac.
2. Use the model to translate monolingual Syriac to synthetic Coptic.
3. Add these synthetic pairs to training data.
4. Re-train. Repeat 2–3 times.

This is the technique that drives much of NLLB’s performance [6] and is essential for genuinely low-resource pairs.

6.6 Distributed Training on 4×A6000

For the recommended setup, use Hugging Face Accelerate [27] with PyTorch FSDP [16] or DeepSpeed ZeRO-2 [15]:

```
from transformers import Seq2SeqTrainingArguments

args = Seq2SeqTrainingArguments(
    output_dir="./checkpoints",
    per_device_train_batch_size=8,
    gradient_accumulation_steps=4,
    num_train_epochs=10,
    learning_rate=1e-4,
    fp16=False, bf16=True, # BF16 on Ampere GPUs
    gradient_checkpointing=True,
    save_steps=1000,
    eval_steps=500,
    deepspeed="ds_config.json", # ZeRO-2 config
)
```

7 Dataset Construction

This section details the data you must collect. Coptic and Syriac are ancient languages with relatively limited digital resources, but a surprising amount is available if you know where to look.

7.1 Parallel Data Sources

Coptic–English / Syriac–English (Bible). The Coptic Bible (Sahidic and Bohairic dialects) and the Syriac Peshitta have been digitized and aligned with English translations. Sources:

- **Sahidica:** Sahidic Coptic New Testament with English alignments (<https://www.biblical-data.org/coptic/>).
- **Peshitta English alignment:** Available through Sedra [25] (<https://sedra.bethmardutho.org/>).
- **eBible.org:** Bible translations in many languages including some Coptic and Syriac variants (<https://ebible.org>).

Expected size: 5K–30K parallel verses per language pair.

Coptic–Greek / Syriac–Greek. Coptic and Syriac New Testaments aligned with the Greek New Testament. The Greek NT is the most aligned text in human history.

- **Coptic Scriptorium** [24]: Provides aligned Coptic–Greek data (<https://copticcriptorium.org>).
- **Center for New Testament Restoration:** Greek–Syriac alignments (<https://greekcntr.org>).

Expected size: 8K–25K verse pairs per language pair.

Patristic literature. Greek Fathers translated into both Syriac and Coptic in late antiquity. Some aligned editions exist.

- **Patrologia Graeca:** Many texts (<https://patristica.net/graeca/>).

- **CMCL (Corpus dei Manoscritti Copti Letterari)**: Coptic literary corpus.
- **Bibliotheca Hagiographica Orientalis**: Syriac hagiographic texts.
General multilingual parallel corpora. Some include Coptic and Syriac:
- **Tatoeba** (<https://tatoeba.org>): Some sentences in both languages with English alignments.
- **OPUS** [23] (<https://opus.nlpl.eu>).
- **Bible Parallel Corpus** (<https://github.com/christos-c/bible-corpus>): Multi-language Bible with both Coptic and Syriac in some forms.
- **WikiMatrix** [22] for related languages where available.
Direct Coptic–Syriac. Very rare, but check:
- Scholarly bilingual editions of Syriac–Coptic monastic texts.
- Any aligned Bible verses (both translate from Greek, so verse-level alignment is implicit).
Expected size: Tens to a few hundred sentences. This is normal.

7.2 Monolingual Data Sources

Coptic monolingual:

- **Coptic Scriptorium** [24] (<https://copticcriptorium.org>): Largest curated digital Coptic corpus, with morphological annotations.
- **Marcion** (<https://github.com/pishoyg/coptic>): Coptic dictionary and texts.
- **Nag Hammadi Library**: Gnostic Coptic texts.
- **Glott500-c** [5] subset for Coptic, if available.

Syriac monolingual:

- **Sedra / Bethmardutho** [25] (<https://sedra.bethmardutho.org>): Comprehensive Syriac digital library.
- **Comprehensive Aramaic Lexicon Project**: Aramaic and Syriac texts (<https://cal.huc.edu>).
- **Syriaca.org** (<https://syriaca.org>): Reference and texts.
- **Open Syriac Corpus**: Various scholarly compilations.

Expected size: 100K–1M sentences total per language, depending on what you can collect.

7.3 Data Preprocessing Pipeline

Once collected, your data needs cleaning and normalization:

1. **Encoding normalization:** Ensure all text is UTF-8 and uses standard Unicode forms (NFC normalization).
2. **Script verification:** Verify Coptic text uses the Coptic Unicode block, not Greek look-alikes; similarly for Syriac.
3. **Sentence segmentation:** Coptic and Syriac have different punctuation conventions; you may need to write language-specific sentence splitters.
4. **Filtering:** Remove very short (< 5 tokens) and very long (> 200 tokens) sentences; remove sentences with high non-target-script content.
5. **Alignment verification:** For parallel data, spot-check that aligned sentence pairs actually correspond.
6. **Train/dev/test split:** Use 90/5/5 or similar. Ensure test set has no overlap with training (e.g., different Bible verses from different sources).

7.4 Evaluation Set Construction

You need a small but high-quality evaluation set. Recommended approach:

- Select 500–1000 sentences from well-aligned Bible texts (e.g., the Gospels) that are present in both Coptic and Syriac.
- Verify alignments manually for a sample of 50–100.
- Use these as a held-out test set. Never use this data for training.
- Report BLEU [17], chrF++ [18], and conduct qualitative analysis on a subset.

For broader evaluation, consider FLORES-200-style evaluation [21] if you can find or commission high-quality translations into both languages.

8 Evaluation and Analysis

8.1 Metrics

- **BLEU** [17]: Standard but limited for morphologically rich languages. Use `sacrebleu` [19] for reproducible scores.
- **chrF++** [18]: Character-level F-score, more appropriate for low-resource and morphologically rich languages.
- **COMET** [20]: Neural metric (requires multilingual sentence encoder that supports your languages; check carefully).
- **Manual qualitative evaluation:** For 20–50 examples, have a human (ideally with knowledge of the languages) rate fluency and adequacy on a 1–5 scale.

8.2 Ablations to Report

Your final report should include ablation studies:

1. **Vocabulary extension vs. not:** How much does adding new tokens matter?
2. **Embedding initialization strategy:** Random vs. character-mean vs. multilingual alignment.
3. **Multi-task training vs. direct only:** How much do auxiliary language pairs help?
4. **Back-translation rounds:** How does performance improve with iterations [10]?
5. **Encoder fine-tuning vs. frozen:** Is full fine-tuning necessary, or is LoRA [13] sufficient? Adapter-based methods like MAD-X [12] are also worth considering.

8.3 Expected Baseline Performance

Realistic expectations for this project:

- BLEU on direct Coptic–Syriac: 5–15 (this is genuinely hard).
- chrF++ scores will be more interpretable: target 30–50.
- Translation quality on religious/Biblical text should be substantially better than on general text.

Do not be discouraged by low BLEU scores; for ancient low-resource pairs, these metrics are notoriously harsh and the meaningful improvement is often qualitative.

9 Project Timeline

A suggested 14-week timeline:

Week	Activity
12 June	Progress Report of the Term project (Full Details)
19 June	Final Presentation of the Term Project
3 July	Submission of the Term Paper together with github link

Table 2: project timeline.

10 Common Pitfalls and Tips

Pitfall 1: Skipping vocabulary verification. Always check tokenization output before training. If you see character-level fragmentation or many UNKs, vocabulary extension is essential.

Pitfall 2: Random embedding initialization. This is the single most common cause of training failure. Always use subword composition initialization or better.

Pitfall 3: Training the decoder before the encoder is adapted. Phase 1 (encoder adaptation) is essential; skipping it means the decoder learns from poor encoder representations.

Pitfall 4: Direct Coptic–Syriac data only. You will not have enough; use auxiliary pairs through Greek and English.

Pitfall 5: Ignoring data quality. Spot-check your parallel data; even Bible alignments can have errors at the sentence level.

Pitfall 6: Catastrophic forgetting. [26] When fine-tuning the entire model, mix in some original-language data to preserve general capabilities. This is a well-known problem in continual learning and applies directly when extending multilingual models.

Pitfall 7: Over-relying on BLEU. BLEU [17] is unreliable for low-resource morphologically rich languages. Use chrF++ [18] as your primary metric, and always include qualitative analysis.

11 Conclusion

This project sits at the intersection of multilingual NLP, transfer learning, and digital humanities. The technical content—vocabulary extension, embedding initialization, cross-attention training, back-translation—covers most of the modern toolkit for low-resource NMT. The ancient languages involved make the project both intellectually engaging and a tangible contribution to scholarship.

The deeper lesson you should take away: **a pre-trained multilingual model is not a black box but a system of components**, each of which can be analyzed, extended, and adapted. Tokenizer, embeddings, attention layers, and decoder architecture are all separately modifiable. Understanding which to change and which to leave alone is what distinguishes effective transfer learning from cargo-cult fine-tuning.

Good luck, and may your gradients flow smoothly.

References

- [1] Benjamin Muller, Antonios Anastasopoulos, Benoît Sagot, and Djamé Seddah. When Being Unseen from mBERT is just the Beginning: Handling New Languages With Multilingual Language Models. In *Proceedings of NAACL-HLT*, 2021. <https://aclanthology.org/2021.naacl-main.38>
- [2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. <https://arxiv.org/abs/1706.03762>
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*, 2019. <https://aclanthology.org/N19-1423>
- [4] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Un-supervised Cross-lingual Representation Learning at Scale (XLM-R). In *Proceedings of ACL*, 2020. <https://aclanthology.org/2020.acl-main.747>
- [5] Ayyoob Imani, Peiqin Lin, Amir Hossein Kargaran, Silvia Severini, Masoud Jalili Sabet, Nora Kassner, Chunlan Ma, Helmut Schmid, André F. T. Martins, François Yvon, and Hinrich Schütze. Glot500: Scaling Multilingual Corpora and Language Models to 500 Languages. In *Proceedings of ACL*, 2023. <https://aclanthology.org/2023.acl-long.61> GitHub: <https://github.com/cisnlp/Glot500>
- [6] NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, et al. No Language Left Behind: Scaling Human-Centered Machine Translation. *Nature*, 2024. <https://www.nature.com/articles/s41586-024-07335-x> GitHub: <https://github.com/facebookresearch/fairseq/tree/nllb>
- [7] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units (BPE). In *Proceedings of ACL*, 2016. <https://aclanthology.org/P16-1162>
- [8] Taku Kudo. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates (UnigramLM). In *Proceedings of ACL*, 2018. <https://aclanthology.org/P18-1007>
- [9] Taku Kudo and John Richardson. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. In *Proceedings of EMNLP: System Demonstrations*, 2018. <https://aclanthology.org/D18-2012> GitHub: <https://github.com/google/sentencepiece>
- [10] Rico Sennrich, Barry Haddow, and Alexandra Birch. Improving Neural Machine Translation Models with Monolingual Data. In *Proceedings of ACL*, 2016. <https://aclanthology.org/P16-1009>
- [11] Yinhan Liu, Jiatao Gu, Naman Goyal, Xian Li, Sergey Edunov, Marjan Ghazvininejad, Mike Lewis, and Luke Zettlemoyer. Multilingual Denoising Pre-training for Neural Machine Translation (mBART). *Transactions of the Association for Computational Linguistics*, 2020. <https://aclanthology.org/2020.tacl-1.47>

- [12] Jonas Pfeiffer, Ivan Vulić, Iryna Gurevych, and Sebastian Ruder. MAD-X: An Adapter-Based Framework for Multi-Task Cross-Lingual Transfer. In *Proceedings of EMNLP*, 2020. <https://aclanthology.org/2020.emnlp-main.617>
- [13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022. <https://arxiv.org/abs/2106.09685>
- [14] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of EMNLP: System Demonstrations*, 2020. <https://aclanthology.org/2020.emnlp-demos.6>
- [15] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *Proceedings of SC20*, 2020. <https://arxiv.org/abs/1910.02054>
- [16] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. In *Proceedings of VLDB*, 2023. <https://arxiv.org/abs/2304.11277>
- [17] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of ACL*, 2002. <https://aclanthology.org/P02-1040>
- [18] Maja Popović. chrF++: Words Helping Character N-grams. In *Proceedings of the Second Conference on Machine Translation*, 2017. <https://aclanthology.org/W17-4770>
- [19] Matt Post. A Call for Clarity in Reporting BLEU Scores. In *Proceedings of the Third Conference on Machine Translation*, 2018. GitHub: <https://github.com/mjpost/sacrebleu>
- [20] Ricardo Rei, Craig Stewart, Ana C Farinha, and Alon Lavie. COMET: A Neural Framework for MT Evaluation. In *Proceedings of EMNLP*, 2020. GitHub: <https://github.com/Unbabel/COMET>
- [21] Naman Goyal, Cynthia Gao, Vishrav Chaudhary, Peng-Jen Chen, Guillaume Wenzek, Da Ju, Sanjana Krishnan, Marc’Aurelio Ranzato, Francisco Guzmán, and Angela Fan. The FLORES-101 Evaluation Benchmark for Low-Resource and Multilingual Machine Translation. *Transactions of the Association for Computational Linguistics*, 2022. <https://github.com/facebookresearch/flores>
- [22] Holger Schwenk, Vishrav Chaudhary, Shuo Sun, Hongyu Gong, and Francisco Guzmán. Wiki-Matrix: Mining 135M Parallel Sentences in 1620 Language Pairs from Wikipedia. In *Proceedings of EACL*, 2021. <https://aclanthology.org/2021.eacl-main.115>
- [23] Jörg Tiedemann. Parallel Data, Tools and Interfaces in OPUS. In *Proceedings of LREC*, 2012. <https://opus.nlpl.eu>

-
- [24] Caroline T. Schroeder and Amir Zeldes. Coptic Scriptorium: An Open Source Platform for Coptic Studies. <https://copticSCRIPTORIUM.org>
 - [25] Bethmardutho: The Syriac Institute. SEDRA: Syriac Electronic Data Research Archive. <https://sedra.bethmardutho.org>
 - [26] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming Catastrophic Forgetting in Neural Networks. In *Proceedings of the National Academy of Sciences*, 2017. <https://arxiv.org/abs/1612.00796>
 - [27] Hugging Face Accelerate Documentation. <https://huggingface.co/docs/accelerate>